Project Deployment Report — Simple Secure Share

What We Set Out to Do

The goal was simple: take a web application called “Simple Secure Share” (a Node.js/Express project) and make it live on the internet — for free — so it could be accessed by anyone through a web link, and so it could be shown off as a real, working project (for example, in a resume or interview).

To do this, two things were needed: 1. A place to store the project’s code safely and permanently — **GitHub** 2. A place to actually run the application so people can visit it — **AWS (Amazon Web Services), using their Free Tier**

How We Did It

Step 1: Backing Up the Code on GitHub

Before putting the app online, the code was uploaded to GitHub. Think of GitHub as a safe, cloud-based storage locker for code — it keeps a history of every change and lets you (or anyone else) redownload the project anytime.

This involved: - Creating a new, empty repository (project folder) on GitHub - Preparing the project files on the computer - Uploading (“pushing”) the code from the computer to that GitHub repository

Why: Having the code on GitHub means it’s never lost, even if something goes wrong on the server later. It also makes future updates much easier — instead of manually copying files, you can just “pull” the latest version.

Step 2: Renting a Free Virtual Computer on AWS

Next, a virtual computer (called an **EC2 instance**) was created using AWS’s Free Tier, which lets you use a small server at no cost for a limited time/usage. This server ran a lightweight version of Linux (Ubuntu).

Why: The app needs to run on a computer that’s always switched on and connected to the internet — your personal laptop can’t do that reliably or safely. AWS gives you exactly that, and the Free Tier makes it free to experiment with.

Step 3: Setting Up the Server

Once the server was ready, a few tools were installed on it: - **Node.js** – the software needed to actually run this type of web application - **Git** – used to download the project code from GitHub onto the server

Why: The server starts out as a blank computer with nothing on it. These tools are the minimum requirements to run the application.

Step 4: Bringing the Code onto the Server

The project code was downloaded from GitHub directly onto the AWS server, and all its required components (dependencies) were installed automatically.

Why: This is the actual “deployment” step — moving your working code from your computer/GitHub onto the live server so it can run there permanently.

Step 5: Configuring the App

A configuration file (`.env`) was created and filled in with the specific settings the app needed to function correctly (like passwords, keys, or settings that shouldn’t be public).

Why: Apps often need private settings that aren’t stored in the public code for security reasons. This step supplies those separately, directly on the server.

Step 6: Keeping the App Running Permanently

A tool called **PM2** was installed and used to start the application in a way that keeps it running even after closing the terminal — and automatically restarts it if the server reboots.

Why: Without this, the app would shut off the moment you disconnect from the server. PM2 makes sure it stays live 24/7.

Step 7: Making the Website Look Professional

Two more things were added: - **Nginx** – lets people visit the site using a normal web address instead of `http://your-ip:3000` (no ugly port numbers) - **DuckDNS + HTTPS (via Let’s Encrypt)** – gave the site an actual readable domain name (`secfileshare.duckdns.org`) and secured it with the padlock icon (🔒) browsers show for safe, encrypted sites

Why: These steps aren’t strictly necessary to make the app “work,” but they make it look and feel like a real, trustworthy, professional website rather than a raw test server.

End Result

The application is now: - ✅ Backed up safely on GitHub - ✅ Running live on an AWS server - ✅ Accessible through a clean web address: **https://secfileshare.duckdns.org** - ✅ Secured with HTTPS (encrypted connection) - ✅ Set up to auto-restart if the server reboots

---

What This Demonstrates

Completing this task, from start to finish, shows hands-on experience with several real-world skills: - Using GitHub for code storage and version control - Setting up and managing a cloud server (AWS) - Basic Linux/server administration - Deploying and running a Node.js application - Configuring domain names and HTTPS security - Keeping an application running reliably (process management)

These are genuinely useful, practical skills relevant to software development, cloud computing, and cybersecurity roles — and this project can be used as a real example of applied experience.

A Few Notes for the Future

- If the server is restarted, the app will come back on automatically (thanks to PM2).
- If the server is stopped and started again, AWS may give it a new internet address (IP) — the DuckDNS domain settings would just need a quick update to match.
- It’s a good habit to regularly save (commit and push) any code changes back to GitHub, so there’s always a backup.